Online Movie /Event Ticket Booking System

Introduction

Movie Magic is a system developed to manage ticket booking for movies and events in an organized and efficient way. It helps users easily explore available movies or events, check schedules and venues, select seats, and book tickets without confusion or delay.

The system handles important aspects such as show scheduling, seat availability, booking details, and payment records. It ensures that all information is stored properly, reducing manual errors and improving accuracy.

In this system, multiple movies or events can be available at different venues and times, and users can make bookings based on their preferences. Each booking can include one or more seats, and all booking and payment details are maintained for future reference.

Overall, Movie Magic provides a simple and reliable way to manage ticket booking activities for both users and administrators.

Objectives

To provide easy ticket booking for movies and events

To allow users to view schedules and select seats

To maintain booking and payment records

To reduce manual effort and errors

Scope

Users can view and book tickets for movies/events

The system manages schedules, seats, and venues

Booking and payment details are stored

Admin can manage movies, events, and bookings

1.1 All the entities involved in the system

- Admin
- Booked seats
- Bookings
- Events
- Movie
- Customer
- Event-bookings
- OTP-verification
- payments
- Shows
- Theater

1.2 List the attributes of each entity

1) Admin

- ADMINID
- Name
- Email
- Password
- Role

2)Booked seats

- id
- booking id
- screen id
- seat no.

3)Bookings

- id
- screen id
- user-id
- no. of seats
- selected seats

4) customer

- user-id
- name
- email
- password
- city

5) Event-bookings

- id
- event-id
- user-id
- user-name
- email

6) Events

- id
- event-name
- category
- city
- venue

7) Movie

- movie-id
- movie-name
- movie-rating
- movie-dimensions
- genre

8)OTP-verifications

- id
- email
- expires-at
- created-at

9) Payments

- id
- booking-id
- amount
- status
- transaction-id

10) Shows

- screen-id
- movie-id
- theater-id
- timings
- show-date

11)Theater

- theater-id
- theater-name
- city
- shows

1.3 Specify the cardinality of each relationship

Relationship	Cardinality
Movie-Shows	1:N
Theater-Shows	1:N
Admin-Customer	1:N
Shows-Bookings	1:N
Customer-Bookings	1:N
Customer-OTP_verification	1:1
Customer- Event_bookings	1:N
Events-Event_bookings	1:N
Bookings-Booked_seats	1:N

Bookings-Payments 1:1

2.1 Represent all entities and relationships in tabular format

Customer

Column	Datatype	Constraint
user_id	SERIAL	PRIMARY KEY
name	VARCHAR(255)	NOT NULL
email	VARCHAR(255)	UNIQUE, NOT NULL
password	VARCHAR(255)	NOT NULL
city	VARCHAR(255)	
contact_no	BIGINT	
provider	VARCHAR(50)	DEFAULT 'local'
profile_pic	TEXT	
is_blocked	BOOLEAN	DEFAULT false

Movie

Column	Datatype	Constraint
movie_id	VARCHAR(255)	PRIMARY KEY
movie_name	VARCHAR(255)	NOT NULL
movie_rating	VARCHAR(50)	
movie_dimensions	VARCHAR(50)	
genre	VARCHAR(255)	
status	VARCHAR(50)	
description	TEXT	
language	VARCHAR(100)	
category	VARCHAR(100)	
duration	VARCHAR(50)	
poster_url	VARCHAR(500)	
poster	VARCHAR(500)	

Shows

Column	Datatype	Constraint
screen_id	VARCHAR(255)	PRIMARY KEY
movie_id	VARCHAR(255)	FOREIGN KEY
theater_id	INTEGER	FOREIGN KEY
timings	TIME	NOT NULL
show_date	DATE	NOT NULL
screen_no	INTEGER	DEFAULT 1
screen_dimensions	VARCHAR(50)	
no_of_seats	INTEGER	DEFAULT 0
selected_seats	JSONB	DEFAULT []

Bookings

Column	Datatype	Constraint
id	SERIAL	PRIMARY KEY
screen_id	VARCHAR(255)	FOREIGN KEY
user_id	INTEGER	
no_of_seats	INTEGER	DEFAULT 0
selected_seats	JSONB	DEFAULT []
price	NUMERIC(10,2)	DEFAULT 0.00
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP
payment_status	VARCHAR(50)	DEFAULT 'Paid'
theater_id	INTEGER	
theater_name	VARCHAR(255)	
movie_id	VARCHAR(255)	
show_date	DATE	
show_time	TIME	

Theater

Column	Datatype	Constraint
theater_id	SERIAL	PRIMARY KEY
theater_name	VARCHAR(255)	NOT NULL
city	VARCHAR(255)	

Booked Seats

Column	Datatype	Constraint
id	SERIAL	PRIMARY KEY
booking_id	INTEGER	FOREIGN KEY
screen_id	VARCHAR(255)	FOREIGN KEY
seat_number	VARCHAR(50)	NOT NULL, UNIQUE (with screen_id)

Payments

Column	Datatype	Constraint
id	SERIAL	PRIMARY KEY
booking_id	INTEGER	FOREIGN KEY
amount	NUMERIC(10,2)	NOT NULL
status	VARCHAR(50)	DEFAULT 'SUCCESS'
transaction_id	VARCHAR(255)	
payment_method	VARCHAR(50)	
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

OTP Verifications

Column	Datatype	Constraint
id	SERIAL	PRIMARY KEY
email	VARCHAR(100)	NOT NULL
otp	VARCHAR(6)	NOT NULL
expires_at	TIMESTAMP	NOT NULL
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP
is_verified	BOOLEAN	DEFAULT false

Events

Column	Datatype	Constraint
id	SERIAL	PRIMARY KEY
event_name	VARCHAR(255)	NOT NULL
category	VARCHAR(50)	NOT NULL
city	VARCHAR(100)	NOT NULL
venue	VARCHAR(255)	NOT NULL
event_date	DATE	NOT NULL
event_time	VARCHAR(20)	NOT NULL
price	INTEGER	NOT NULL

Event Bookings

Column	Datatype	Constraint
id	SERIAL	PRIMARY KEY
event_id	INTEGER	FOREIGN KEY
user_id	INTEGER	NOT NULL
user_name	VARCHAR(255)	
email	VARCHAR(255)	
phone	VARCHAR(20)	
tickets_count	INTEGER	NOT NULL
selected_seats	TEXT[]	
amount	INTEGER	NOT NULL
payment_status	VARCHAR(20)	DEFAULT 'Paid'
booking_date	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

3.1 Convert the unnormalized travel reservation data into normalized relations.

Unnormalized Form (UNF)

In unnormalized form, data contains repeating groups and redundant attributes.

Bookings (UNF)

id

user_id

screen_id

no_of_seats

selected_seats (Seat1, Seat2, Seat3…) (multiple values in one field)

Event Bookings (UNF)

event_id

user_id

user_name (redundant)

email (redundant)

Problems:

Multiple values in one column (selected_seats)

Duplicate user data (user_name, email)

Data redundancy and inconsistency

- ## First Normal Form (1NF)

Rule: Remove repeating groups, make all values atomic.

Changes Made:

selected_seats removed from Bookings

New table created: Booked Seats

Bookings (1NF)

id

user_id

screen_id

no_of_seats

Booked Seats (1NF)

id

booking_id

screen_id

seat_no (one seat per row)

Now all values are atomic (1NF achieved)

- ## Second Normal Form (2NF)

Definition:

A table is in 2NF if it is in 1NF and all non-key attributes are fully dependent on the primary key (no partial dependency).

In this database:

All tables have a single primary key, so partial dependency does not occur.

Every non-key attribute depends completely on its primary key.

Example: Booked Seats

Primary Key: id

Attribute: seat_no

- **seat_no depends fully on id (not on part of a key)**
- **So, no partial dependency exists**

Conclusion (2NF)

The database satisfies 2NF

No changes were required because full dependency already exists

- ## Third Normal Form (3NF)

Rule: Remove transitive dependency (non-key should not depend on another non-key)

Changes Made:

Removed user_name and email from Event Bookings

Event Bookings (3NF)

event_id

user_id

- **User details are now stored only in Customer table**

- **Conclusion**

UNF had redundancy and multi-valued attributes

1NF removed repeating groups (created Booked Seats)

2NF ensured full dependency (already satisfied)

3NF removed redundant attributes (cleaned Event Bookings)

Final database is normalized up to 3NF

3.2 Write the SQL commands to create the normalized tables

```
-- ======================
-- CUSTOMER
-- ======================
CREATE TABLE customer (
   user_id SERIAL PRIMARY KEY,
   name VARCHAR(255) NOT NULL,
   email VARCHAR(255) UNIQUE NOT NULL,
   password VARCHAR(255) NOT NULL,
   city VARCHAR(255),
   contact_no BIGINT,
   provider VARCHAR(50) DEFAULT 'local',
   profile_pic TEXT,
   is_blocked BOOLEAN DEFAULT false
);

-- ======================
-- MOVIE
-- ======================
```

```
CREATE TABLE movie (
    movie_id VARCHAR(255) PRIMARY KEY,
    movie_name VARCHAR(255) NOT NULL,
    movie_rating VARCHAR(50),
    movie_dimensions VARCHAR(50),
    genre VARCHAR(255),
    status VARCHAR(50),
    description TEXT,
    language VARCHAR(100),
    category VARCHAR(100),
    duration VARCHAR(50),
    poster_url VARCHAR(500),
    poster VARCHAR(500)
);
```

```
-- =======================
-- THEATER
-- =======================
CREATE TABLE theater (
   theater_id SERIAL PRIMARY KEY,
   theater_name VARCHAR(255) NOT NULL,
   city VARCHAR(255)
);


-- =======================
-- SHOWS
-- =======================
CREATE TABLE shows (
   screen_id VARCHAR(255) PRIMARY KEY,
   movie_id VARCHAR(255),
   theater_id INTEGER,
   timings TIME NOT NULL,
show_date DATE NOT NULL,
   screen_no INTEGER DEFAULT 1,
   screen_dimensions VARCHAR(50),
   no_of_seats INTEGER DEFAULT 0,
```

```
    selected_seats JSONB DEFAULT '[]',


    FOREIGN KEY (movie_id) REFERENCES movie(movie_id) ON DELETE CASCADE,
    FOREIGN KEY (theater_id) REFERENCES theater(theater_id) ON DELETE CASCADE
);
```

```
-- ======================
-- BOOKINGS
-- ======================
CREATE TABLE bookings (
    id SERIAL PRIMARY KEY,
    screen_id VARCHAR(255),
    user_id INTEGER,
    no_of_seats INTEGER DEFAULT 0,
    selected_seats JSONB DEFAULT '[]',
    price NUMERIC(10,2) DEFAULT 0.00,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    payment_status VARCHAR(50) DEFAULT 'Paid',
    theater_id INTEGER,
    theater_name VARCHAR(255),
    movie_id VARCHAR(255),
    show_date DATE,
    show_time TIME,

    FOREIGN KEY (screen_id) REFERENCES shows(screen_id) ON DELETE CASCADE
);
```

```
-- ======================
-- BOOKED SEATS
-- ======================
CREATE TABLE booked_seats (
    id SERIAL PRIMARY KEY,
    booking_id INTEGER,
    screen_id VARCHAR(255),
    seat_number VARCHAR(50) NOT NULL,


    FOREIGN KEY (booking_id) REFERENCES bookings(id) ON DELETE CASCADE,
    FOREIGN KEY (screen_id) REFERENCES shows(screen_id) ON DELETE CASCADE,
    UNIQUE (screen_id, seat_number)
);
```

```
-- ======================
-- PAYMENTS
-- ======================
CREATE TABLE payments (
    id SERIAL PRIMARY KEY,
    booking_id INTEGER,
    amount NUMERIC(10,2) NOT NULL,
    status VARCHAR(50) DEFAULT 'SUCCESS',
    transaction_id VARCHAR(255),
    payment_method VARCHAR(50),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    FOREIGN KEY (booking_id) REFERENCES bookings(id) ON
DELETE CASCADE
);
-- ======================
-- OTP VERIFICATION
-- ======================
CREATE TABLE otp_verifications (
    id SERIAL PRIMARY KEY,
    email VARCHAR(100) NOT NULL,
```

```
    otp VARCHAR(6) NOT NULL,
    expires_at TIMESTAMP NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    is_verified BOOLEAN DEFAULT false
);
```

```sql
-- ======================
-- EVENTS
-- ======================
CREATE TABLE events (
    id SERIAL PRIMARY KEY,
    event_name VARCHAR(255) NOT NULL,
    category VARCHAR(50) NOT NULL,
    city VARCHAR(100) NOT NULL,
    venue VARCHAR(255) NOT NULL,
    event_date DATE NOT NULL,
    event_time VARCHAR(20) NOT NULL,
    price INTEGER NOT NULL,
    total_seats INTEGER NOT NULL,
    booked_seats JSONB DEFAULT '[]',
    image_url TEXT,
    description TEXT,
    status VARCHAR(20) DEFAULT 'Active',
    has_seat_map BOOLEAN DEFAULT false,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```sql
-- ======================
-- EVENT BOOKINGS
-- ======================
CREATE TABLE event_bookings (
    id SERIAL PRIMARY KEY,
    event_id INTEGER,
    user_id INTEGER NOT NULL,
    user_name VARCHAR(255),
    email VARCHAR(255),
    phone VARCHAR(20),
    tickets_count INTEGER NOT NULL,
    selected_seats TEXT[],
    amount INTEGER NOT NULL,
    payment_status VARCHAR(20) DEFAULT 'Paid',
    booking_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    FOREIGN KEY (event_id) REFERENCES events(id)
);
```